

Felix Moitzi
Daniel Siegfried Knapp
Zlata Zelenska

Integration of HYPR Learning Algorithm into non-spiking Recurrent Neural Networks

PROJECT

Master's degree programme: Software Engineering and Management / Computer Science

Supervisors

Robert Legenstein
Maximilian Baronig
Yeganeh Bahariasl

Institute of Machine Learning and Neural Computation
Graz University of Technology

Graz, 04 2026

Abstract

Recurrent neural networks (RNNs) remain competitive models for sequence learning, but their standard training procedure, backpropagation through time (BPTT), is sequential, offline, and memory intensive on long inputs. Our study answers whether Hybrid Propagation (HYPR), recently introduced for recurrent spiking neural networks, can be transferred to non-spiking gated recurrent architectures. We adapt HYPR to long short-term memory (LSTM) networks and gated recurrent units (GRUs) by introducing a gate-aware formulation of cell computations and eligibility tensors. This allows us to combine forward propagation of local eligibility information with segment-wise parallel gradient accumulation in standard gated RNNs. We then compare this implementation with BPTT and evaluate it against matched baselines on sequence classification tasks.

Our analysis shows that the integration itself is feasible but also highlights a central trade-off: gating improves expressivity while making the saving-regime memory required by HYPR more demanding. Therefore, in our tested case HYPR is best understood as an efficiency- and constraint-driven alternative rather than a universal replacement for BPTT in non-spiking gated RNNs. It is preferable to choose HYPR when one is bounded in available memory, needs online operation, or depends on long sequence processing, while BPTT remains the stronger default when final predictive performance and optimization robustness are the main priorities.

Language: Python

Libraries: JAX · Flax · Hydra

Keywords: recurrent neural networks · BPTT · HYPR · LSTM · GRU · online learning

Contents

1	Introduction	4
2	Background & Related Work	5
3	Implementation & Methodology	10
4	Results	15
5	Conclusion	30
	Bibliography	31

1 Introduction

Recurrent neural networks are a natural choice for sequence data because they update an internal state while processing one input element after another. This makes them suitable for classification, prediction, and control problems in which the current output depends not only on the current input, but also on information that has to be retained over time [6, 4]. At the same time, this sequential structure makes training difficult. When gradients are computed with BPTT, the recurrent computation has to be unrolled over time and differentiated in reverse order. As a result, both runtime and memory usage grow with the effective sequence length, and exact online learning is not possible [9, 1].

This bottleneck motivates the search for alternatives that preserve the modelling advantages of RNNs while relaxing the practical constraints of BPTT. One family of approaches changes the model class itself, for example by moving to attention-based architectures or to sequence models with more parallelizable state updates. Another family keeps the recurrent architecture but changes the learning algorithm. HYPR belongs to the second group. Instead of replacing recurrent computation, it combines approximate forward propagation of local credit information with segment-wise parallel gradient accumulation aiming to balance online capability, bounded memory, and parallel execution [1].

The central question of this seminar project is whether HYPR remains meaningful and practical when transferred from spiking recurrent networks to standard non-spiking gated architectures. We focus on LSTM and GRU models because they are strong recurrent baselines and because their gating mechanisms introduce an informative stress test for HYPR: the models are expressive, but their internal structure makes gradient bookkeeping more involved. In this sense, the project is not only an empirical comparison between HYPR and BPTT, but also an implementation study of how a learning rule originally formulated for spiking neurons can be integrated into gated recurrent cells.

Our contribution therefore lies in three points. First, we place HYPR into the broader landscape of recurrent learning algorithms by relating it to BPTT, exact online gradient learning via Real-Time Recurrent Learning (RTRL), and the eligibility-trace perspective of e-prop. Second, we describe the concrete mathematical and implementation changes that were needed to support gate-aware HYPR in LSTM and GRU cells. Third, we structure the experimental comparison to BPTT on sequential benchmarks.

2 Background & Related Work

Generic recurrent sequence model

We begin with a generic recurrent model defined on an input sequence $x_{1:T} = (x_1, \dots, x_T)$, where $x_t \in \mathbb{R}^{d_x}$. Let $s_t \in \mathbb{R}^{d_s}$ denote the recurrent state at time step t , let $\hat{y}_t \in \mathbb{R}^{d_y}$ denote the model output, and let θ collect the trainable parameters.

A broad class of recurrent models can be written as

$$I_t = W_x x_t + W_s s_{t-1} + b, \quad (2.1)$$

$$s_t = f_\theta(s_{t-1}, I_t), \quad (2.2)$$

$$\hat{y}_t = g_\theta(s_t). \quad (2.3)$$

Here I_t is a vector-valued input current (or preactivation) that is an affine combination of the current input and information carried by the previous recurrent state. In architectures where only part of the recurrent state is exposed to this affine map, Eq. (2.1) should be read only as a general rule. In particular, for the LSTM the affine gate preactivations depend on h_{t-1} , while the cell state c_{t-1} enters separately through the state update.

The training objective is usually an accumulated sequence loss,

$$L(\theta) = \sum_{t=1}^T \ell_t(\hat{y}_t, y_t), \quad (2.4)$$

where y_t denotes the target associated with time step t or, in sequence classification, the label that supervises the sequence as a whole. The central technical challenge is that the contribution of a parameter at time step t may only become visible much later in the sequence. This is the temporal credit assignment problem [9, 10].

LSTM and GRU models are special cases of this general framework. Their state update functions are designed so that information can be preserved or discarded in a controlled way, which reduces, but does not completely remove the training difficulties of recurrent models [6, 4]. In particular, the gradient still has to pass through a chain of recurrent Jacobians across time.

For the gradient derivations in following sections we use θ to denote the parameters of the recurrent state update f_θ . If the readout g_θ also contains trainable parameters, they contribute an additional direct term which is omitted here for notational clarity.

Backpropagation Through Time

BPTT computes exact gradients for a finite sequence by unfolding the recurrent computation over time and applying reverse-mode automatic differentiation to the resulting computation graph [9, 2]. Throughout this section, gradients of scalar losses with respect to vector states are written as row-vector adjoints, so Jacobians multiply on the right.

For the generic model above, define the state error signal

$$\delta_t := \frac{\partial L}{\partial s_t} \in \mathbb{R}^{1 \times d_s}, \quad (2.5)$$

By the chain rule, the backward recursion is

$$\delta_t = \frac{\partial \ell_t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial s_t} + \delta_{t+1} \frac{\partial s_{t+1}}{\partial s_t}. \quad (2.6)$$

The parameter gradient is

$$\frac{\partial L}{\partial \theta} = \sum_{t=1}^T \delta_t \frac{\partial s_t}{\partial \theta} \Big|_{\text{direct}}. \quad (2.7)$$

In this form the dependence on time is explicit: BPTT requires access to the intermediate states of the unfolded computation and therefore uses memory that scales with the sequence length [9, 1]. From an implementation point of view, BPTT is exact and conceptually clean, but the backward pass remains sequential over time, which limits parallelism on long recurrent traces.

Why gradients vanish or explode

The backward recursion above contains products of recurrent Jacobians. Expanding the recursion shows that the influence of a later loss term on an earlier state contains factors of the form

$$\frac{\partial s_\tau}{\partial s_t} = \prod_{k=t+1}^{\tau} \frac{\partial s_k}{\partial s_{k-1}}. \quad (2.8)$$

If these Jacobian products typically shrink vectors, gradient signals decay toward zero. If they typically expand vectors, gradients can grow rapidly and become unstable. This is the classical vanishing/exploding gradient problem in recurrent learning [6].

This matters here for two reasons. First, it motivates gated architectures such as LSTMs and GRUs, whose additive state paths and multiplicative gates make long-range information flow easier to preserve [6, 4]. Second, it clarifies why online alternatives to BPTT are difficult: any algorithm that aims to capture long-range temporal credit assignment must somehow represent or approximate these same chains of Jacobians.

Exact online learning and the RTRL viewpoint

If BPTT is the exact offline solution, RTRL is the classical exact online solution. Instead of storing the unfolded computation graph and differentiating it backward after the full sequence has been processed, RTRL propagates the total derivative of the recurrent state with respect to the parameters forward in time [10]. Define

$$P_t := \frac{ds_t}{d\theta}. \quad (2.9)$$

Here, P_t summarizes how every component of the current state depends on every parameter after all recurrent interactions up to time step t have been taken into account. Using the state update $s_t = f_\theta(s_{t-1}, x_t)$, the sensitivity recursion is

$$P_t = \frac{\partial f_\theta(s_{t-1}, x_t)}{\partial \theta} + \frac{\partial f_\theta(s_{t-1}, x_t)}{\partial s_{t-1}} P_{t-1}. \quad (2.10)$$

The first term is the direct parameter dependence of the current state update, while the second term propagates the previously accumulated sensitivity through the recurrent dynamics. The loss gradient w.r.t parameters θ at time step t is

$$\nabla_\theta \ell_t = \frac{\partial \ell_t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial s_t} P_t. \quad (2.11)$$

RTRL enables true online learning and avoids memory that grows with the total sequence length. Its limitation is computational rather than conceptual: for general recurrent networks, the carried sensitivity object P_t is very large, and the resulting complexity makes exact online learning impractical except in very small or highly structured settings [10, 1].

Eligibility traces and e-prop

E-prop addresses this limitation by keeping the forward, online part of RTRL while replacing the full nonlocal sensitivity, saving which is infeasible, with a local approximation that can be computed per neuron or per synapse [3]. Its characteristic form factorizes the gradient into a learning signal and an eligibility trace.

Because eligibility traces are naturally defined per neuron or per synapse, we now switch from the layer-level state notation s_t to neuron-level notation such as s_j^t and parameter indices θ_{ji} . For a parameter θ_{ji} associated with neuron or synapse (j, i) , the approximation is written as

$$\frac{dL}{d\theta_{ji}} \approx \sum_{t=1}^T L_j^t e_{ji}^t, \quad (2.12)$$

where L_j^t is a task-dependent learning signal and e_{ji}^t is a local eligibility trace that summarizes how past activity makes the parameter relevant for the current state.

The eligibility trace itself obeys a forward recursion,

$$e_{ji}^t = \frac{\partial s_j^t}{\partial \theta_{ji}} \Big|_{\text{direct}} + \frac{\partial s_j^t}{\partial s_j^{t-1}} e_{ji}^{t-1}. \quad (2.13)$$

Thus, e-prop preserves neuron-internal temporal credit assignment through the local state dynamics, but avoids carrying the full cross-neuron recurrent dependency structure of RTRL.

The critical approximation is that gradient pathways through the recurrent interactions of other neurons are dropped. E-prop therefore retains a forward, online learning rule, but no longer computes the exact gradient of the full recurrent network [3].

HYPR as a hybrid algorithm

HYPR was introduced for recurrent spiking neural networks as a hybrid between approximate online learning and segment-wise parallel approximate backpropagation [1]. Conceptually, it combines two ideas: it keeps the forward propagation of eligibility information known from e-prop, but reorganizes the computations over short subsequences so that part of the gradient accumulation can be carried out in parallel within each segment.

To make this more explicit, consider a single neuron i and define

$$A_i^t := \frac{\partial s_i^t}{\partial s_i^{t-1}}, \quad D_i^t := \frac{\partial s_i^t}{\partial \theta_i} \Big|_{\text{direct}}. \quad (2.14)$$

Here, A_i^t is the local state-transition Jacobian and D_i^t is the direct same-step parameter Jacobian of the state update. The eligibility recursion can then be written as

$$e_i^t = A_i^t e_i^{t-1} + D_i^t. \quad (2.15)$$

This is the forward-propagated quantity that carries long-range temporal influence across segments.

Now partition a long sequence into K consecutive segments of equal length λ . For one such segment, let e_i^0 denote the incoming eligibility from the previous segment. Define the cumulative state-transition matrix

$$\Phi_i^{\lambda:t} := A_i^\lambda A_i^{\lambda-1} \dots A_i^t. \quad (2.16)$$

Using this notation, the eligibility at the end of the current segment can be written as

$$e_i^\lambda = D_i^\lambda + \sum_{t=1}^{\lambda-1} \Phi_i^{\lambda:t+1} D_i^t + \Phi_i^{\lambda:1} e_i^0. \quad (2.17)$$

This expression shows explicitly how direct state-to-parameter contributions from the current segment, together with the eligibility carried from earlier segments, are forward-projected to the segment boundary.

HYPR complements this forward propagation with a segment-local reverse accumulation. Let q_i^t denote a backward coefficient inside the segment, defined recursively by

$$q_i^\lambda = \frac{\partial \ell_\lambda}{\partial s_i^\lambda}, \quad q_i^t = q_i^{t+1} A_i^{t+1} + \frac{\partial \ell_t}{\partial s_i^t}, \quad t = \lambda - 1, \dots, 1, \quad \text{with } q_i^0 := q_i^1 A_i^1 \quad (2.18)$$

The cumulative approximate gradient contribution of the segment can then be written as

$$\tilde{\nabla}_{\theta_i} L^{(1:\lambda)} = q_i^0 e_i^0 + \sum_{t=1}^{\lambda} q_i^t D_i^t. \quad (2.19)$$

At the full-sequence level, HYPR combines the segment contributions as

$$\tilde{\nabla}_\theta L = \sum_{k=1}^K \tilde{\nabla}_\theta L^{(k)}. \quad (2.20)$$

The coefficients q_i^t perform a local backward-style accumulation inside the current segment, while the eligibility state e_i^0 carries temporal credit information across segment boundaries. The segments are therefore not independent; they remain coupled through the propagated eligibility state.

For the present project, the most important HYPR-specific design variable is the segmentation granularity. More chunks increase the scope for parallel execution, but they also make the approximation more aggressive because fewer cross-time interactions are captured inside any single segment-local backward computation. This trade-off becomes especially important in gated architectures, where multiple gates and internal state variables increase the amount of memory needed to save eligibility matrix and derivatives required by the algorithm.

3 Implementation & Methodology

This chapter outlines the key steps of the project, detailing the reasoning behind each decision, and the specific methodologies used. It provides an overview of the implementation and evaluation process.

Model architectures

We focus on two standard gated RNN architectures:

- Long Short-Term Memory [6].
- Gated Recurrent Unit [4].

In both cases, the recurrent computation is performed at the cell level, and hidden layers are stacked into a multi-layer recurrent model with a linear readout layer on top.

Since the HYPR training algorithm operates on the level of recurrent state transitions and their derivatives, it was necessary to implement both architectures explicitly at the cell level in JAX. Instead of using pre defined recurrent layers, individual LSTM and GRU cells were implemented and later combined into full recurrent layers. This allows direct access to internal states, gate activations, and state derivatives, which are required for the computation of HYPR.

To transfer notation from previous chapter: the recurrent state becomes $s_t = h_t$ for the GRU and $s_t = (h_t, c_t)$ for the LSTM, where h_t is the recurrently communicated hidden/output signal and c_t is the internal cell state.

LSTM

For an input x_t , previous hidden state h_{t-1} , and previous cell state c_{t-1} , the standard LSTM update is

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i), \quad (3.1)$$

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f), \quad (3.2)$$

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c), \quad (3.3)$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o), \quad (3.4)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad (3.5)$$

$$h_t = o_t \odot \tanh(c_t). \quad (3.6)$$

The additive update of c_t is the characteristic feature that improves long-range gradient flow. In the implementation, the recurrently communicated hidden/output signal h_t and the internal cell state c_t are tracked explicitly so that the state derivatives required by HYPR can be computed in a consistent way.

GRU

For the GRU, the update equations are

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z), \quad (3.7)$$

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r), \quad (3.8)$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h), \quad (3.9)$$

$$h_t = (1 - z_t) \odot \tilde{h}_t + z_t \odot h_{t-1}. \quad (3.10)$$

Compared with LSTM, the GRU has no separate cell state, which reduces the dimensionality of the recurrent state but still retains multiplicative control over information flow through the update and reset gates. This makes the GRU a useful comparison point: it is structurally simpler than the LSTM, but still more involved than a vanilla recurrent cell.

HYPR integration into gated RNNs

The core implementation challenge is that HYPR does not only require the forward pass of a recurrent cell. It also requires access to state derivatives, gate-specific Jacobians, and eligibility tensors that match the internal structure of the cell. This is straightforward for simple recurrent units, but less so for gated architectures, where the state update is assembled from several interacting nonlinear subcomputations, i.e. separate gates.

Explicit gate axis

To make the gated recurrent computations compatible with HYPR, the input projection of each recurrent layer was written in a gate-aware form. Instead of producing a single preactivation vector per hidden unit, the implementation produces a tensor

$$I_t \in \mathbb{R}^{B \times N \times G}, \quad (3.11)$$

where B is the batch size, N the number of hidden units, and G the number of gate channels. In our setting, $G = 4$ for the LSTM and $G = 3$ for the GRU.

At the layer level, we first compute a concatenated gate preactivation

$$z_t = W_x x_t + W_h h_{t-1} + b, \quad z_t \in \mathbb{R}^{B \times (NG)}, \quad (3.12)$$

and then reshape it into

$$I_t = \text{reshape}(z_t, B, N, G) \in \mathbb{R}^{B \times N \times G}. \quad (3.13)$$

Here, x_t is the current external input and h_{t-1} is the recurrently communicated hidden/output signal from the previous time step.

The explicit gate axis is important for HYPR because different gate channels contribute differently to the state transition and therefore to the corresponding eligibility tensors.

State representation and shape stability

A second implementation issue concerned shape stability of the recurrent carry. In the gate-aware formulation, the preactivations at time step t are stored in a tensor

$$I_t \in \mathbb{R}^{B \times N \times G},$$

where B is the batch size, N the number of hidden units, and G the number of gate channels. A naive implementation based on splitting this tensor can leave each gate slice with an unnecessary singleton dimension, for example $(B, N, 1)$ instead of (B, N) . Such layout changes make the recurrent carry more unreliable in code because the state representation is then no longer uniform across time steps.

To avoid this, we access gate channels by direct indexing,

$$I_t^{(g)} = I_t[\dots, g], \quad g = 0, \dots, G - 1,$$

so that each gate-specific tensor has the stable shape (B, N) . This keeps the state update equations shape-consistent across all time steps and simplifies the use of automatic differentiation in JAX.

This choice is particularly important for HYPR. In our implementation, eligibility tensors are propagated together with the ordinary recurrent state through the scan-based subsequence computation. Consistent tensor layouts are therefore required not only for correct state updates, but also for correct eligibility propagation and efficient compiled execution. Direct gate indexing ensures that the recurrent state and the eligibility state remain compatible throughout the scan.

Eligibility tensors and memory trade-off

HYPR removes the explicit dependence of memory usage on the total sequence length, but it does not make memory use negligible. The algorithm stores eligibility tensors whose shape depends on the number of trainable parameters, the hidden-state dimension, and the number of gates. In gated architectures this cost is noticeably higher than in simpler recurrent cells (G for gated is 4 or 3 and 1 for spiking architectures). Consequently, our practical comparison between HYPR and BPTT is not only about time versus memory, but also about what kind of memory is used. BPTT stores sequence-local activations for the backward pass; HYPR stores carried eligibility information that grows with model

structure, especially with gate dimension.

This point is crucial for interpreting the experiments. In non-spiking gated RNNs, the main implementation burden of HYPR arises not from recurrence alone, but from the combination of recurrence, gating, and eligibility-tensor saving workflow.

Datasets

For this project, we used three sequential datasets that differ in temporal structure and task difficulty. Sequential MNIST (sMNIST), that requires the model to classify handwritten digits from pixel sequences, and is a standard benchmark. For the experiments with the sMNIST dataset we applied the *sum_of_softmax* loss aggregation and prediction mode.

Furthermore, the SHD (Spiking Heidelberg digits) dataset, developed in 2020 by a team of researchers at Heidelberg University primarily motivated by the fact that datasets for benchmarking in between SNN implementations are scarce, is utilized in order to evaluate the performance of the HYPR training algorithm in contrast to the BPTT approach. The dataset consists of 10.000 high-quality recordings of spoken digits (zero to nine), generated by 12 different speakers, 50% of whom are male and 50% are female, in both German and English language [5].

In the SHD experiments we applied also the *sum_of_softmax* loss aggregation and prediction mode, similar to the configuration with the sMNIST dataset.

Additionally, we generated the dataset for the pathfinder experiments, by modifying the *snakes2_wrapper.py* from the pathfinder repository [8] for 32x32 output frames, as described in the LRA documentation [7]. After generating the pathfinder images and metadata, the files are compressed into difficulty-level-separated *.pickle* files, which can further be used as input data for the HYPR, as well as for the BPTT implementation, provided in the project repository. The experiments with this dataset specifically should enable us to assess the ability of the model under test to capture long range dependencies and try HYPR on a larger input data.

Along with the pathfinder experiments we applied the *per_timestep_cross_entropy* and also tried using *last_timestep_cross_entropy* as a loss aggregation method. When it comes to the prediction mode setting, we applied the *mean_over_sequence* mode, as well as the *last_timestep_prediction* configuration.

Experiments

Each experiment was implemented in a separate configuration file, defining the setup for a specific combination of model, dataset, training method and hyper parameters. This structure allows a clear separation between experiments and allowed reproducible comparisons between HYPR and BPTT, without modifying the underlying codebase.

Hyperparameters

In this section we explain the key hyperparameters we choose for our model, reasoning every selection. In machine learning, selecting the appropriate hyperparameters is often challenging, as there rarely is a single analytical configuration that guarantees optimal performance. Instead, hyperparameter tuning typically requires a combination of empirical testing and relying on configurations that have been proven to be effective in other similar tasks. To find an effective combination, with respect to the technical boundaries of the cluster we have been provided with, we tested several configurations using a grid search strategy, and empirically chose the best model in terms of performance.

- **Learning rate** – controls convergence speed and training stability.
- **Batch size** – influences gradient quality and memory usage.
- **Hidden size** – determines model capacity.
- **Number of layers** – affects model depth.
- **Epochs** – controls training duration.
- **HYPR - Number of Chunks** – determines the temporal partitioning of sequences in HYPR.

This configuration choice was tailored to our model’s requirements and hardware limitations, allowing for optimal performance within the available resources.

4 Results

In the course of this work, we not only implemented the LSTM and GRU model to work with the HYPR training algorithm, but also evaluated the performance of the models on the datasets, described in section 3. For each of the models, a systematic grid-search is conducted over a dataset-specific hyperparameter grid, leveraging both, the BPTT and the HYPR training algorithm.

NOTE: Due to the time constraint, given by the computational cluster which limits the job duration to 4 days, the hyperparameters are primarily selected to provide comparative metrics between the applied models (per dataset) with different training algorithms (BPTT and HYPR), rather than achieving the highest validation/test accuracy. Nevertheless, we also include obtained accuracy and loss metrics, especially for different epoch-configurations (including 50 and 100 epoch training setups), using the hyperparameters from the best run, obtained during the grid search procedure.

Besides comparing the models itself, we also aim to get insights about the model performance if we choose HYPR as the main training algorithm and increase the number-of-chunks parameter value.

SHD Experiment

For the experiments with the SHD (Spiking Heidelberg digits) dataset we chose the following parameter-grid:

Hyperparameter	Values
Learning Rate	{0.002, 0.001, 0.0005}
Batch Size	{64, 128, 256}
Hidden Size	{128, 256, 512}
Number of Layers	{1, 2, 3}
Epochs	{10, 20, 30}
Number of Chunks	{1, 2}

Table 4.1: Grid-search hyperparameter configuration SHD

The application of this parameter-grid during the grid-search procedure yields 486 parameter-combinations for the experiments with HYPR and 243 individual configurations for the experiments with the BPTT.

LSTM model performance - SHD

Using **BPTT**, the LSTM achieves validation accuracy of **0.9963** with the following hyperparameter configuration:

LR	Batch Size	Hidden Size	Nr. of Layers	Epochs
0.002	256	256	3	30

Table 4.2: BPTT - Hyperparameter configuration LSTM

On the test set, this hyperparameter configuration achieves an accuracy of **88.38%**.

The following table describes the model’s performance on different epoch-configurations, including the results obtained during grid search, extended by tests with the same hyperparameter configuration as in the best run for epoch values 50 and 100.

Number Epochs	10	20	30	50	100
Test Accuracy (%)	86.3074	90.5035	88.3834	89.8852	87.9417
Validation Accuracy	0.9840	0.9950	0.9963	0.9914	0.9914

Table 4.3: BPTT’s performance metrics across different epoch configurations

The following configuration lead to the best test accuracy overall during the grid search procedure with validation accuracy of **0.9791** and test accuracy of **91.43%**.

LR	Batch Size	Hidden Size	Nr. of Layers	Epochs
0.002	64	256	3	20

Table 4.4: BPTT - Hyperparameter configuration LSTM best test accuracy

Using **HYPR**, the LSTM achieves a validation accuracy of **0.9914** with the following hyperparameter configuration:

LR	Batch Size	Hidden Size	Nr. of Layers	Epochs	Nr. of Chunks
0.001	64	512	3	30	2

Table 4.5: HYPR - Hyperparameter configuration LSTM

On the test set, this hyperparameter configuration achieves an accuracy of **84.01%**.

NOTE: Even though the number of chunks is set to 2 in this configuration, the run with number of chunks set to 1 yields a comparable result with validation accuracy value of **0.9901** and even better test accuracy of **84.84%**. Notably, this results in a worse test accuracy score when compared with the best test accuracy (ranked by validation

accuracy) obtained when running BPTT instead of HYPR with **88.38%**.

The performance on different epoch configurations is described in the table below:

Number Epochs	10	20	30	50	100
Test Accuracy (%)	85.95	82.86	84.01	85.37	85.37
Validation Accuracy	0.9828	0.9901	0.9914	0.9901	0.9901
Number of chunks	2	2	2	2	2

Table 4.6: HYPR’s performance metrics across different epoch configurations

The configuration stated in table 4.7 lead to the best test accuracy overall during the grid search procedure with validation accuracy of **0.9914** and test accuracy of **89.39%**.

LR	Batch Size	Hidden Size	Nr. of Layers	Epochs	Nr. of Chunks
0.0005	64	512	3	30	2

Table 4.7: HYPR - Hyperparameter configuration LSTM best test accuracy

Since we are also interested in the influence of the number-of-chunks parameter on the validation/test accuracy, we run the experiment with the same hyperparameter-configuration as in the best run, obtained during grid-search, and iterate through different number-of-chunks parameter (epochs=30):

Number Chunks	1	2	5	10	25
Test Accuracy (%)	55.03	84.01	83.78	81.66	83.65
Validation Accuracy	0.9877	0.9914	0.9889	0.9901	0.9877

Table 4.8: Performance metrics across different numbers of chunks

The result shows that increasing the number of chunks does not necessarily lead to a decrease in validation/test accuracy. Apparently the HYPR training algorithm is very sensitive to hyperparameter configurations across different number-of-chunk parameter values. Therefore, if the number of chunks is not constrained to be fixed (e.g. because of memory limits), hyperparameter evaluation has to be performed for each number-of-chunks parameter value.

If we take a look at the result from the grid search, where the number-of-chunk parameter was set to 1, the run with the best validation accuracy of **0.9901** yields a test accuracy of **84.84%** when training 30 epochs (which is higher than the test accuracy with number-of-chunk parameter set to 2).

GRU model - SHD

Using **BPTT**, the GRU achieves validation accuracy of **0.9950** with the following hyperparameter configuration:

LR	Batch Size	Hidden Size	Nr. of Layers	Epochs
0.0005	64	512	3	30

Table 4.9: BPTT - Hyperparameter configuration GRU

On the test set, this hyperparameter configuration achieves an accuracy of **86.70%**.

The following table describes the model’s performance on different epoch-configurations from the grid search, extended by test runs with epoch parameters 50 and 100, based on the hyperparameter configuration of the best run (ranked by validation accuracy).

Number Epochs	10	20	30	50	100
Test Accuracy (%)	84.36	89.17	86.70	84.76	88.56
Validation Accuracy	0.9816	0.9938	0.9950	0.9938	0.9938

Table 4.10: BPTT’s performance metrics across different epoch configurations

The following configuration lead to the best test accuracy overall during the grid search procedure with validation accuracy of **0.9803** and test accuracy of **89.62%**.

LR	Batch Size	Hidden Size	Nr. of Layers	Epochs
0.002	128	256	1	20

Table 4.11: BPTT - Hyperparameter configuration GRU best test accuracy

Using **HYPR**, the GRU achieves validation accuracy of **0.9889** with the following hyperparameter configuration:

LR	Batch Size	Hidden Size	Nr. of Layers	Epochs	Nr. of Chunks
0.0005	64	512	3	30	1

Table 4.12: HYPR - Hyperparameter configuration GRU

On the test set, this hyperparameter configuration achieves an accuracy of **85.68%**. The performance on different epoch configurations is described in the table below:

Number Epochs	10	20	30	50	100
Test Accuracy (%)	83.70	85.29	85.68	85.68	85.68
Validation Accuracy	0.9742	0.9840	0.9889	0.9889	0.9889
Number of chunks	1	1	1	1	1

Table 4.13: HYPR’s performance metrics across different epoch configurations

The following configuration (table 4.14) leads to the best test accuracy overall during the grid search procedure with validation accuracy of **0.9718** and test accuracy of **87.72%**.

LR	Batch Size	Hidden Size	Nr. of Layers	Epochs	Nr. of Chunks
0.002	128	512	3	20	1

Table 4.14: HYPR - Hyperparameter configuration GRU best test accuracy

The following table describes the test- and validation accuracy of the GRU model, featuring the HYPR training algorithm with different number-of-chunks parameter values (hyper-param configuration of best grid-search run as stated in table 4.12):

Number Chunks	1	2	5	10	25
Test Accuracy (%)	85.68	85.51	85.60	84.84	82.90
Validation Accuracy	0.9889	0.9852	0.9889	0.9828	0.9767

Table 4.15: Performance metrics across different numbers of chunks

This result perfectly shows, that with an increasing number of chunks, the accuracy in general and specifically the test accuracy decreases.

For being able to directly compare the performance of the GRU with the LSTM model when utilizing the HYPR training algorithm, we search for the best run with **hypr_args.num_chunks** set to **2**. The following configuration yields validation accuracy of **0.9852** with test accuracy of **84.36%**.

LR	Batch Size	Hidden Size	Nr. of Layers	Epochs	Nr. of Chunks
0.001	64	512	2	30	2

Table 4.16: HYPR - Hyperparameter configuration GRU - direct comparison

This configuration yields the following accuracy metrics across epochs:

Number Epochs	10	20	30	50	100
Test Accuracy (%)	83.70	85.29	84.36	84.36	84.36
Validation Accuracy	0.9742	0.9840	0.9852	0.9852	0.9852
Number of chunks	2	2	2	2	2

Table 4.17: HYPR’s performance metrics across different epoch configurations

NOTE: The accuracy metrics for epoch configuration 50 and 100 are obtained from test runs, featuring the same hyperparameter configuration like the run with 30 training epochs. For epoch configurations 10, 20 and 30, the results are obtained from the grid search output.

SMNIST Experiment

For the SMNIST experiments, we applied the same hyperparameter grid as with the SHD experiments:

Hyperparameter	Values
Learning Rate	{0.002, 0.001, 0.0005}
Batch Size	{64, 128, 256}
Hidden Size	{128, 256, 512}
Number of Layers	{1, 2, 3}
Epochs	{10, 20, 30}
Number of Chunks	{1, 2}

Table 4.18: Grid-search hyperparameter configuration SMNIST

LSTM model performance - SMNIST

Using **BPTT**, the lstm achieves a validation accuracy of **0.9888** with the following hyperparameter configuration:

LR	Batch Size	Hidden Size	Nr. of Layers	Epochs
0.0005	128	512	2	30

Table 4.19: BPTT - Hyperparameter configuration LSTM

On the test set, this hyperparameter configuration achieves an accuracy of **98.89%**. Similarly to the experiments with the SHD dataset, we also include obtained metrics from epoch configurations, not present in the parameter grid of the grid-search procedure. The hyperparameters chosen during this extra-runs are equal to those, obtained in the grid search (of the best run).

Number Epochs	10	20	30	50	100
Test Accuracy (%)	98.38	98.75	98.89	98.86	98.95
Validation Accuracy	0.9826	0.9868	0.9888	0.9898	0.9919

Table 4.20: BPTT’s performance metrics across different epoch configurations

The following configuration lead to the best test accuracy overall during the grid search procedure with validation accuracy of **0.9864** and test accuracy of **98.94%**.

LR	Batch Size	Hidden Size	Nr. of Layers	Epochs
0.001	64	128	2	30

Table 4.21: BPTT - Hyperparameter configuration LSTM best test accuracy

Using **HYPR**, the LSTM achieves validation accuracy of **0.9676** with the following hyperparameter configuration:

LR	Batch Size	Hidden Size	Nr. of Layers	Epochs	Nr. of Chunks
0.001	64	128	3	30	2

Table 4.22: HYPR - Hyperparameter configuration LSTM

On the test set, this hyperparameter configuration achieves an accuracy of **96.6%**. Importantly, this combination of hyperparameters also serves as the hyperparameter configuration for the global maximum test accuracy.

The performance on different epoch configurations is described in the table below:

Number Epochs	10	20	30	50	100
Test Accuracy (%)	93.73	96.37	96.60	97.00	96.93
Validation Accuracy	0.9433	0.9646	0.9676	0.9713	0.9739
Number of chunks	2	2	2	2	2

Table 4.23: HYPR’s performance metrics across different epoch configurations

The configuration stated in table 4.22 also achieved the best test accuracy across all iterations during the grid search.

Table 4.24 describes the accuracy of the LSTM model on the SMNIST dataset with different number-of-chunks parameter values (but with same hyperparameter configuration stated in table 4.22).

Number Chunks	1	2	4	8	16	28
Test Accuracy (%)	93.39	96.60	90.91	92.11	96.83	11.35
Validation Accuracy	0.9350	0.9676	0.9129	0.9271	0.9678	0.1115

Table 4.24: Performance metrics across different numbers of chunks

Similarly to the experiments with the SHD dataset, the results show that proper hyperparameter tuning is essential, when using different number of chunks. While with the SHD experiments the number-of-chunks parameter value of 1 lead to a significant decrease in test accuracy, when it comes to the experiments with the SMNIST dataset, the number-of-chunks parameter value of 28 leads to a serious decrease in both, validation and test accuracy.

When doing hyperparameter grid search with fixed number of chunks (28) for 30 epochs, the following configuration yields the best validation accuracy of **0.9563** and test accuracy of **95.21%**, which is just slightly lower than the test accuracy when specifically doing hyperparameter tuning with number-of-chunks set to 2 with **96.60%**:

LR	Batch Size	Hidden Size	Nr. of Layers	Epochs	Nr. of Chunks
0.002	64	128	2	30	28

Table 4.25: HYPR - Hyperparameter configuration LSTM - 28 Chunks

GRU model - SMNIST

Using **BPTT**, the GRU achieves a validation accuracy of **0.9876** with the following hyperparameter configuration:

LR	Batch Size	Hidden Size	Nr. of Layers	Epochs
0.002	128	128	3	30

Table 4.26: BPTT - Hyperparameter configuration GRU

On the test set, this hyperparameter configuration achieves an accuracy of **98.58%**.

The performance across different epoch configurations is stated in table 4.27. Similar to the experiments with the LSTM model, the stated accuracy metrics are obtained from the grid-search result for epoch configuration 10, 20 and 30. The accuracy metrics for the test runs with epochs 50 and 100 are generated using the same hyperparameter configuration as in the best run of the grid-search procedure (ranked by validation accuracy).

Number Epochs	10	20	30	50	100
Test Accuracy (%)	98.24	98.79	98.58	97.70	97.70
Validation Accuracy	0.9796	0.9864	0.9876	0.9801	0.9801

Table 4.27: BPTT’s performance metrics across different epoch configurations

The following configuration lead to the best test accuracy overall during the grid search procedure with validation accuracy of **0.9864** and test accuracy of **98.79%**.

LR	Batch Size	Hidden Size	Nr. of Layers	Epochs
0.001	64	256	2	20

Table 4.28: BPTT - Hyperparameter configuration GRU best test accuracy

Using **HYPR**, the GRU achieves validation accuracy of **0.9534** with the following hyperparameter configuration:

LR	Batch Size	Hidden Size	Nr. of Layers	Epochs	Nr. of Chunks
0.001	64	256	3	30	1

Table 4.29: HYPR - Hyperparameter configuration GRU

On the test set, this hyperparameter configuration achieves an accuracy of **95.03%**. The performance on different epoch configurations is described in the table below:

Number Epochs	10	20	30	50	100
Test Accuracy (%)	90.15	94.62	95.03	95.03	95.03
Validation Accuracy	0.8995	0.9469	0.9534	0.9534	0.9534
Number of chunks	1	1	1	1	1

Table 4.30: HYPR’s performance metrics across different epoch configurations

The configuration stated in table 4.29 also lead to the best test accuracy overall during the grid search procedure (validation accuracy of **0.9534** and test accuracy of **95.03%**).

Table 4.31 describes the test- and validation accuracy of the GRU model, featuring the HYPR training algorithm with different number-of-chunks parameter values (evaluated by training 30 epochs on hyper-params from best run in grid-search as stated in table 4.29):

Number Chunks	1	2	4	8	16	28
Test Accuracy (%)	95.03	11.35	14.56	81.60	13.85	17.15
Validation Accuracy	0.9534	0.1115	0.1476	0.8193	0.1456	0.1706

Table 4.31: Performance metrics across different numbers of chunks

We again conclude, that proper hyperparameter evaluation needs to be done when applying different number-of-chunks parameter values. In fact, when taking a look at the grid search output, the GRU achieves very competitive accuracy metrics (when compared to the experiment with the LSTM model) with properly selected hyperparameter settings (**hypr_args.num_chunks** set to **2**) as it can be seen in table 4.33.

For comparison purposes, we also include the obtained results for the GRU model on the SMNIST dataset with the same HYPR-related parameter configuration (number of chunks). The grid-search with number-of-chunks set to 2 yielded the following hyperparameter configuration with validation accuracy of **0.9406%** and test accuracy of **93.84%**.

LR	Batch Size	Hidden Size	Nr. of Layers	Epochs	Nr. of Chunks
0.001	64	128	3	30	2

Table 4.32: HYPR - Hyperparameter configuration GRU - direct comparison

This configuration yields the following accuracy metrics across epochs:

Number Epochs	10	20	30	50	100
Test Accuracy (%)	87.71	89.69	93.84	93.82	94.05
Validation Accuracy	0.8723	0.8923	0.9406	0.9458	0.9458
Number of chunks	2	2	2	2	2

Table 4.33: HYPR’s performance metrics across different epoch configurations

NOTE: Similar to the tests with GRU on the SHD dataset, the accuracy metrics for epoch configuration 50 and 100 are obtained from test runs, featuring the same hyperparameter configuration as in the run with epochs set to 30. For epoch configurations 10, 20 and 30, the results are obtained from the grid search output.

Pathfinder-E Experiment

The hyperparameter grid used in the grid-search procedure on the Pathfinder-E dataset is stated below.

Note: we only change the epoch-configuration in order to reduce grid-search evaluation time. The reason of choosing the value 50 comes from the past observation that this amount of training epochs leads to very high test and validation accuracy in parallel. Similarly, we do not include the obtained accuracy metrics for other epoch parameter values in table 4.36 and 4.38.

Hyperparameter	Values
Learning Rate	{0.002, 0.001, 0.0005}
Batch Size	{64, 128, 256}
Hidden Size	{128, 256, 512}
Number of Layers	{1, 2, 3}
Epochs	{50}
Number of Chunks	{1, 2}

Table 4.34: Grid search hyperparameter configuration Pathfinder-E

LSTM model performance - Pathfinder-E

Using **BPTT**, the LSTM achieves a validation accuracy of **0.5190** with the following hyperparameter configuration:

LR	Batch Size	Hidden Size	Nr. of Layers	Epochs
0.002	128	256	1	50

Table 4.35: BPTT - Hyperparameter configuration LSTM

On the test set, this hyperparameter configuration yields accuracy of 50.4%. Table 4.36 describes the obtained accuracy metrics on different epochs. In this case (pathfinder experiments), we use a unique set (table 4.35) of hyperparameters for the tests along all epoch configurations.

Number Epochs	10	20	30	50	100
Test Accuracy (%)	50.05	50.70	49.15	50.40	49.15
Validation Accuracy	0.5135	0.5139	0.5200	0.5190	0.5200

Table 4.36: BPTT's performance metrics across different epoch configurations

It can be observed, that the validation- as well as the test accuracy fluctuate around 50% across all epoch configurations. For the pathfinder experiment, which represents a binary classification task, this means that the model does not learn correctly. This gives rise to the assumption that the vanilla LSTM and GRU models we implemented are not able to make meaningful predictions, at least not with BPTT. Since BPTT served as a robust and hard to beat baseline in our experiments, the probability of HYPR to increase predictive performance is very low.

Nevertheless, we also run experiments with HYPR, resulting in the following hyperparameter configuration, yielding validation accuracy of **0.5240** and test accuracy of **51.85%**.

LR	Batch Size	Hidden Size	Nr. of Layers	Epochs	Nr. of Chunks
0.001	128	256	1	50	2

Table 4.37: HYPR - Hyperparameter configuration LSTM

Across different epoch configurations, those hyperparameters yield the following validation- and test accuracy metrics:

Number Epochs	10	20	30	50	100
Test Accuracy (%)	50.05	50.05	50.05	51.85	50.05
Validation Accuracy	0.5135	0.5135	0.5135	0.5240	0.5135
Number of chunks	2	2	2	2	2

Table 4.38: HYPR's performance metrics across different epoch configurations

Similar to the experiments with BPTT, here it also seems like the LSTM is not able to generate meaningful predictions with the pathfinder dataset.

GRU model - Pathfinder-E

Similarly to the experiments with SMNIST and SHD, we also include the GRU's model performance into our comparison:

LR	Batch Size	Hidden Size	Nr. of Layers	Epochs
0.002	128	256	2	50

Table 4.39: BPTT - Hyperparameter configuration GRU

Number Epochs	10	20	30	50	100
Test Accuracy (%)	50.05	50.20	50.05	51.05	50.05
Validation Accuracy	0.5135	0.5135	0.5135	0.5210	0.5135

Table 4.40: BPTT's performance metrics across different epoch configurations

Analogous to the experiments before, we also applied HYPR:

LR	Batch Size	Hidden Size	Nr. of Layers	Epochs	Nr. of Chunks
0.0005	128	256	1	50	1

Table 4.41: HYPR - Hyperparameter configuration GRU

Number Epochs	10	20	30	50	100
Test Accuracy (%)	50.65	50.20	50.05	50.05	47.70
Validation Accuracy	0.4895	0.4915	0.5305	0.5305	0.5220
Number of chunks	1	1	1	1	1

Table 4.42: HYPR’s performance metrics across different epoch configurations

For the sake of completeness, we also include the accuracy metrics for HYPR’s number-of-chunks set to 2 for comparison purposes (hyperparameter evaluation was done in grid-search):

LR	Batch Size	Hidden Size	Nr. of Layers	Epochs	Nr. of Chunks
0.002	256	256	3	50	2

Table 4.43: HYPR - Hyperparameter configuration GRU (2 chunks)

Number Epochs	10	20	30	50	100
Test Accuracy (%)	49.65	50.05	49.55	49.65	50.10
Validation Accuracy	0.5279	0.5135	0.5180	0.5279	0.5135
Number of chunks	2	2	2	2	2

Table 4.44: HYPR’s performance metrics across different epoch configurations (comparison purposes)

Experiments’ Outcomes

We now give a short overview about the concrete differences in performance when we compare the LSTM and the GRU model on the same dataset and consequently evaluate the loss in performance when applying HYPR instead of the standard BPTT training algorithm. Importantly, we also include the best test accuracy obtained during the grid search procedure in brackets ”()”.

Model comparison / HYPR vs. BPTT

In terms of performance between models on the SHD dataset, we observe that the LSTM model achieves slightly higher test accuracy **88.28%** (**91.43%**) than the GRU model **86.70%** (**89.62%**) when the BPTT training algorithm is applied. When it comes to HYPR, the same pattern continues (at least for the max. obtained test accuracy). The LSTM, featuring HYPR, achieves a test accuracy of **84.01%** (**89.39%**) while the GRU achieves **85.68%** (**87.72%**). Since our training/evaluation pipeline currently does not support early stopping, the configuration obtained does not necessarily lead to maximization of test accuracy in a global manner (hyperparameter space). Since the hyperparameter grid includes 10-step epoch configurations, it is possible that the

model "over-trains", resulting in a perfect fit on the validation data but in total lower generalization.

When we compare the performance on the SMNIST dataset, we again observe that GRU's performance in terms of test accuracy **98.58% (98.79%)** is slightly worse than LSTM's result with **98.89% (98.94%)** when applying BPTT. With HYPR, the same pattern continues: The LSTM achieves **96.6% (96.6%)** in test accuracy, while the GRU's result is **95.03% (95.03%)**.

In general, it can be argued that HYPR serves as a slightly worse performing replacement for the BPTT method. The experiments show that the test accuracy is slightly worse in all tested scenarios, when HYPR is applied, but competitive predictive output can still be achieved.

HYPR - number of chunks

Increasing the number of chunks results in a decrease in predictive performance, at least to some degree. The experiment on the SHD dataset with the GRU model serves as a perfect example for this behaviour (see table 4.15).

Furthermore, the tests with the SMNIST dataset, featuring the LSTM model (also applies to the GRU model), show that when we do hyperparameter tuning for a specific number-of-chunks parameter value, other number-of-chunks parameter configurations do not necessarily cope well with the same model-specific (and training-related) parameter configuration. While the test-accuracy obtained with 2 chunks was at **96.60%**, after doing hyperparameter-tuning (grid-search), the setting with 28 chunks could also achieve **95.21%**.

Pathfinder dataset

Since the experiments with the pathfinder dataset suggested, that the model does not learn at all, we identified four probable causes for this behavior:

- the model is not able to model pathfinder data at all (independently of training time and input data resolution)
- the model is constrained by the input data (32x32 images are hard to read even for humans)
- training time is too short (number of training epochs has to be increased)
- the prediction-pipeline is faulty

Even if the model is constrained by a suboptimal hyperparameter configuration, with long training time and input data of high quality, the loss during training should become

smaller along with longer training time and the validation/test accuracy should slightly increase on the 50% test accuracy, the regime currently dominating our accuracy metrics. If this is not the case, we can conclude that the model is either not able to capture the information content present in the pathfinder experiments or something is wrong with the pre-processing/data pipeline/loss generation or optimization.

This is why we double checked dataset generation, the part which was actively adopted during this study (other pathfinder experiments have already been implemented and executed), and we come to the conclusion that the dataset is not faulty. Furthermore, we iterated through different dataset-related configurations and tried different methods of loss aggregation (`per_timestep_cross_entropy`, `last_timestep_cross_entropy`) and prediction mode (`mean_over_sequence`, `last_timestep_prediction`), not resulting in a significant improvement on the already existing results. Furthermore, we tried to train on a smaller dataset, trying to overfit/generate training accuracy beyond the 50% regime, also leading to the conclusion that the model does not necessarily learn stable patterns and cannot at least produce a decent fit on small input data (accepting low generalization on purpose). Furthermore, in order to make sure that input data is granular enough (32x32 images are hard to classify even for the human eye), we adopted the already existing pipeline for 128x128 input images and trained for 300 epochs straight on the LSTM model with BPTT and HYPR training algorithm. The results show that also with higher resolution images the classification task does not yield high validation- or test accuracy, and the results do not significantly differ from the results, obtained with 32x32 input data:

Accuracy Metrics	BPTT	HYPR
Test Accuracy (%)	50.60	50.60
Validation Accuracy	0.5070	0.5070

Table 4.45: Pathfinder results on 128x128 input data (LSTM, 300 epochs)

This result gives rise to the assumption, that the models implemented (LSTM, GRU), in their current state, are not able to provide meaningful and correct classification output for the pathfinder experiment in a consistent manner.

5 Conclusion

This seminar project investigated whether HYPR can be transferred from recurrent spiking neural networks to standard non-spiking gated RNNs. The answer is positive at the implementation level: HYPR can be integrated into LSTM and GRU models without redesigning the recurrent architecture itself. But once gating is introduced, the state representation, gate-wise Jacobians, and eligibility tensors all become more structured. This structure demands more computational power and has to be handled explicitly in the implementation.

From the perspective of learning dynamics, the main picture is a trade-off rather than a clear dominance result. BPTT remains the strongest baseline when exact gradient quality and robust optimization are the primary objectives. HYPR becomes apparent main choice when we care about constraints that BPTT handles weakly: bounded memory with respect to total sequence length, partial parallelization over subsequences, and compatibility with online or streaming regimes. In other words, the relevance of HYPR depends less on whether it can outperform BPTT in a standard benchmark table, and more on specific real-world application setting.

Bibliography

- [1] Maximilian Baronig, Yeganeh Baharinasl, Ozan Özdenizci, and Robert Legenstein. *A Scalable Hybrid Training Approach for Recurrent Spiking Neural Networks*. 2025. arXiv: 2506.14464 [cs.NE]. URL: <https://arxiv.org/abs/2506.14464>.
- [2] Atilim Gunes Baydin, Barak A. Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. “Automatic Differentiation in Machine Learning: A Survey”. In: *Journal of Machine Learning Research* 18.153 (2018), pp. 1–43.
- [3] Guillaume Bellec, Franz Scherr, Anand Subramoney, Elias Hajek, Darjan Salaj, Robert Legenstein, and Wolfgang Maass. “A Solution to the Learning Dilemma for Recurrent Networks of Spiking Neurons”. In: *Nature Communications* 11 (2020), p. 3625. DOI: 10.1038/s41467-020-17236-y.
- [4] Kyunghyun Cho, Bart van Merriënboer, Caglar Gülçehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation”. In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Doha, Qatar: Association for Computational Linguistics, 2014, pp. 1724–1734. DOI: 10.3115/v1/D14-1179. URL: <https://aclanthology.org/D14-1179/>.
- [5] Benjamin Cramer, Yannik Stradmann, Johannes Schemmel, and Friedemann Zenke. “The heidelberg spiking data sets for the systematic evaluation of spiking neural networks”. In: *IEEE Transactions on Neural Networks and Learning Systems* 33.7 (2020), pp. 2744–2757.
- [6] Sepp Hochreiter and Jürgen Schmidhuber. “Long Short-Term Memory”. In: *Neural Computation* 9.8 (1997), pp. 1735–1780. DOI: 10.1162/neco.1997.9.8.1735.
- [7] *Long-Range Arena*. <https://github.com/google-research/long-range-arena>. Accessed: 2026-03-22.
- [8] *Pathfinder*. <https://github.com/drewlinsley/pathfinder>. Accessed: 2026-03-22.
- [9] Paul J. Werbos. “Backpropagation Through Time: What It Does and How to Do It”. In: *Proceedings of the IEEE* 78.10 (1990), pp. 1550–1560. DOI: 10.1109/5.58337.
- [10] Ronald J. Williams and David Zipser. “A Learning Algorithm for Continually Running Fully Recurrent Neural Networks”. In: *Neural Computation* 1.2 (1989), pp. 270–280.